

Computational Intelligence and Safe Reduction of Test Suite

Aftab Ali Haider¹, Aamer Nadeem², Shahzad Rafiq³

Center for Software Dependability
Mohammad Ali Jinnah University (MAJU),
Islamabad, Pakistan

¹aftab775@yahoo.com, ²anadeem@jinnah.edu.pk, ³shahzad@jinnah.edu.pk

Abstract— Systems are frequently regression tested during the maintenance phase due to corrective, preventive, adaptive or perfective actions. Regression testing is used to prevent the undesirable effects of these changes on the previously tested version. Due to these changes, new test cases become part of the test suite making it huge and inefficient for ‘retest all’ strategy. The ultimate solution of this problem is optimization or reduction of the test suite. Computational Intelligence (CI) based approaches like evolutionary computation, fuzzy logic, neural networks and swarm optimization have been used for test suite reduction. Optimization approaches reduce the test suite by compromising its safety. Ideally optimization of test suite must guarantee safe reduction. In this work, we have optimized the test suite using some CI based approaches and then analysed the test suite for ‘safe reduction’. Safe reduction can be gauged using control flow graphs. Test cases of optimal solutions were traversed on these graphs. We found that these solutions partially cover control flow graph. This showed that optimal solutions returned by CI based approaches except fuzzy logic are not safe and will be inadequate for regression testing.

Index Terms— test suite optimization, regression testing, multi objective optimization, safe reduction, fuzzy logic, evolutionary algorithms, swarm optimization, computational intelligence.

I. INTRODUCTION

Software testing is an important activity in software development life cycle [1]. Once the system is developed and implemented, there occur frequent requirement changes in the software [2]. These changes may influence the previously tested software [3-4, 8]. In order to ensure that there is no ripple effect of these changes, system is regression tested [2]. A regression test suite comprises the test cases previously passed by the system software [4]. Additional test cases are written for the changed part of the system [6]. Once the system qualifies, additional test cases also become part of the regression test suite [2]. Systems keep on undergoing frequent requirement changes due to corrective, preventive, perfective or adaptive actions [1]. These changes tremendously increase the size of test suite [6]. The safest regression testing is ‘retest all’ strategy but in some cases, it may spend months to execute the entire test suite [2]. In order to perform the regression testing within the desired budget by spending lesser time requires finding a representative subset of test cases for the

entire test suite [4]. Any reduction in test suite may render test suite ‘unsafe’ and ‘inadequate’ for testing [2]. This subset must guarantee ‘safe reduction’. The process of finding a representative subset is termed as optimization [7].

Computational Intelligence (CI) [9] comprises neural networks, fuzzy systems, evolutionary computation, swarm optimization and artificial immune system. CI based approaches except artificial immune system has been used for test suite optimization [4, 10-11].

In Genetic Algorithms (GA) [12], a random population is generated and is assigned fitness using fitness function. Parents are selected from ranked population using selection approaches and undergo mutation or cross over. New generation produced is compared with the original and if found optimal, is selected as final population otherwise same process is followed till finding of best generation. Regression test suites have been optimized using genetic algorithms and its variants [13-16]. These approaches implement single objective optimization using pure essence of genetic algorithms [6]. Multi objective genetic algorithms have been used by Yoo and Harman [5] for test suite optimization.

In swarm optimization, Bee Colony Optimization (BCO) [17], Ant Colony Optimization (ACO) [18] and Particle Swarm Optimization (PSO) [19] have been implemented to optimize test suite [20-24]. These approaches optimize test suite for single objective except PSO that has been used for multi objective test suite optimization [25]. Hybrid approach using neural networks and genetic algorithms has been used for optimization [26]. Neural network have been used to classify the feature vectors in clusters and then software version is analysed for selection of most suitable cluster for test cases [27].

Fuzzy expert systems have been proposed for optimization of test suite [28-30]. Significance of using fuzzy logic in test suite optimization problem has been outlined by Kumar et al. [31-32]. Fuzzy clustering approach used to classify the similar test cases for test suite selection has been proposed by Kumar et al. [33]. Fuzzy based test suite optimization using fuzzy and iterative fuzzy through tuning of membership functions has been proposed by Haider et al. [4]. Fuzzy based dynamic optimization has been proposed [11].

After reviewing this literature, we have found that safe reduction has not been considered in these approaches except fuzzy based expert systems that can provide safe reduction after human intervention. However, we can evaluate CI based approaches using safe reduction criterion. This work is concerned with CI based approaches and applicability of these approaches for safe reduction of test suite.

Rest of the paper comprises following sections; in section II, we have briefly explained safe reduction for test suite optimization problem. Section III introduces the case study to analyse the multi objective test suite optimization approaches for academic testing problem of quadratic equation. In section IV, we have concluded our work and have discussed future directions.

II. SAFE REDUCTION

Whenever, system evolves due to changes in requirements, it becomes mandatory to perform regression testing [3]. Since ‘Retest all’ regression testing is expensive and time consuming, optimization of test suite is only solution [6]. With the optimization of test suite, safety and precision of test suite are compromised [2]. Thus it is important to reduce the test suite such that safe reduction is guaranteed [4]. Contemporary approaches are efficient to handle multi objective test suite optimization but question arises ‘are these safe?’ Any reduction that may exclude critical test cases is inadequate and may cause catastrophe and is meaningless. Worth of an optimization approach in regression testing is determined by the capability of the approach to include all those test cases in the reduced test suite that could find faults [2]. Safe reduction is characterized using inclusiveness, precision, generality and efficiency [2]. These parameters can be used to determine the safe reduction of test suite.

None of the optimization approach has built in safe reduction procedure. However, it is possible to ensure safe reduction using some methodology based on safe reduction criterion. However, it is possible to achieve safe reduction by applying safe reduction adequacy criteria [36] on final output of these approaches CI based approaches, like multi- objectives GA and PSO, generate Pareto Front [34, 35]. On Pareto optimal solutions, heuristic like weighted sum or center of gravity can be used to achieve results on which safe reduction adequacy criterion can be applied [4].

We have implemented the CI based approaches on quadratic equation problem with the intention to apply adequacy criteria on final optimal set achieved after applying heuristics.

III. CASE STUDY

We have selected quadratic equation problem for this work described in [2]. Optimization objectives for our research work include control flow adequacy coverage, execution time and fault detection where control flow coverage for each test case is determined using decision and condition coverage criteria described in [4]. Execution time, coverage, fault detection and path traversed by the test suite of quadratic equation problem extracted from [4] are given in Table I.

TABLE I
OPTIMIZATION PARAMETERS FOR CASE STUDY [4]

Sr #	Path	Time	Coverage	Faults
1	4	0.003704	0.497	2
2	6	0.001796	0.653	1
3	2	0.002822	0.8	2
4	3	0.002508	0.647	2
5	3	0.001686	0.647	2
6	3	0.001752	0.647	2
7	1	0.006329	0.317	2
8	4	0.002782	0.497	2
9	3	0.002552	0.647	3
10	3	0.002549	0.647	2
11	3	0.001967	0.647	2
12	5	0.003691	0.5	2
13	1	0.003644	0.317	4
14	4	0.00813	0.497	2
15	2	0.002361	0.8	1
16	2	0.002569	0.8	2
17	3	0.00177	0.647	2
18	3	0.003093	0.647	3
19	1	0.007112	0.317	4

Objective functions for fault detection, control flow coverage and execution time are given as under in Eq. 1-3.

$$\text{Function}_{\text{faults}}(x) = \text{Faults (i) / Seeded Faults} \quad \text{for } i = 1 \text{ to } n \quad (1)$$

$$\text{Function}_{\text{coverage}}(x) = \text{Coverage (i) / Adeq. Coverage} \quad \text{for } i = 1 \text{ to } n \quad (2)$$

$$\text{Function}_{\text{time}}(x) = \text{Time (i) / Reference Time} \quad \text{for } i = 1 \text{ to } n \quad (3)$$

In the following section, implementation of CI based approaches on test suite optimization problem is discussed.

A. Multi Objective Genetic Algorithm (MOGA)

Fonesca and Fleming [37] introduced multi objective genetic algorithm based on ranked fitness assignment method in 1993. We implemented MOGA using fitness function defined in Eq 1-3 in Matlab® [38]. Pareto front generated is given in [4]. Optimal front for our problem is encircled in Fig.1.

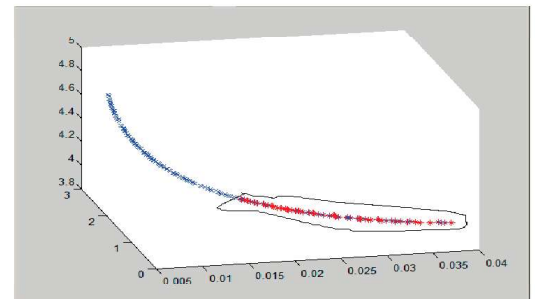


Fig.1. Pareto Front for MOGA Approach [4]

We extracted optimal subset by selecting those solutions on optimal front that are within the range of initial test cases given

in Table I. Thus we have analyzed the decision space and selected relevant solutions and used control flow graph for further analysis given in [4]. Their exist possibility to apply other heuristics like weighted sum, center of gravity etc. Three possible solutions are shown in Fig. 2, 3 & 4.

First possible solution is encircled in Fig.2. We have selected those test cases that fall within this encircled region and are given in Table-II.

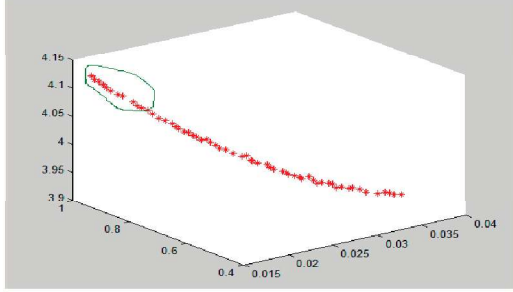


Fig.2. Optimal Solution-I for MOGA Approach

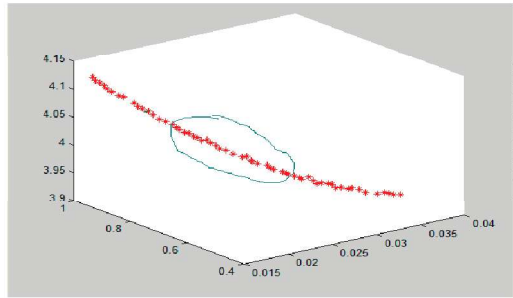


Fig.3. Optimal Solution-II for MOGA Approach

The resultant optimal suite has reduced by approximately 37 percent. Unfortunately, the reduced test suite does not contain Path 1, 4 and 5 test cases. This observation reflects that resultant suite is not safe because the faults related to these paths will not be revealed by the reduced test suite. Secondly, the resultant test suite is not precise because it contains redundant test cases that reveal the faults for the same path.

TABLE II
OPTIMAL SUITE USING MOGA
(SOLUTION-I)

Sr#	Path	Time	Coverage	Faults
2	6	0.001796	0.653	1
3	2	0.002822	0.8	2
4	3	0.002508	0.647	2
5	3	0.001686	0.647	2
6	3	0.001752	0.647	2
9	3	0.002552	0.647	3
10	3	0.002549	0.647	2
11	3	0.001967	0.647	2
15	2	0.002361	0.8	1
16	2	0.002569	0.8	2
17	3	0.00177	0.647	2
18	3	0.003093	0.647	3

Second optimal solution for the MOGA approach is encircled in Fig.3. Test cases that represent this region are

given in Table-III. Upper limits are 0.002 for time and 0.79 for coverage and lower limits are 0.0037 for time and 0.4 for coverage parameters.

These results depict the same scenario as in solution-I where the resultant suite has reduced but was neither safe nor precise. Similarly, the resultant suite does not reveal the faults for the path 2, 5 and 6 although the suite has been reduced by approximately 68 percent.

TABLE III
OPTIMAL SUITE USING MOGA
(SOLUTION-II)

Sr#	Path	Time	Coverage	Faults
4	3	0.002508	0.647	2
8	4	0.002782	0.497	2
9	3	0.002552	0.647	3
10	3	0.002549	0.647	2
13	1	0.003644	0.317	4
18	3	0.003093	0.647	3

Third optimal solution for the MOGA approach is shown in Fig.4. Test cases representing this region are given in Table-IV.

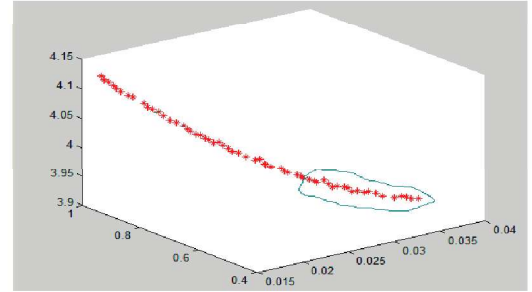


Fig.4. Optimal Solution-III for MOGA Approach

The resultant test suite has reduced by approximately 73 percent but does not reveal faults for the paths 1, 2 and 6.

TABLE IV
OPTIMAL SUITE USING MOGA
(SOLUTION-III)

Sr#	Path	Time	Coverage	Faults
1	4	0.003704	0.497	2
8	4	0.002782	0.497	2
12	5	0.003691	0.5	2
13	1	0.003644	0.317	4
18	3	0.003093	0.647	3

Thus, it can be inferred that MOGA cannot give safe and precise test suite reduction for our problem because it has not been able to traverse all paths. This approach has been successful to explore all five seeded faults.

B. Non-dominated Sorted Genetic Algorithm (NSGA-II)

NSGA-II is a ranked based on non dominated sorting approach and uses crowding distance and elitism to generate the Pareto front [39]. We implemented NSGA-II for our problem with objective function defined in Eq. 1-3. Pareto front generated using NSGA-II approach is given in [4].

Through careful analysis of resultant solution, we have found three possible solutions suitable for our problem. These solutions are represented in Fig.5.

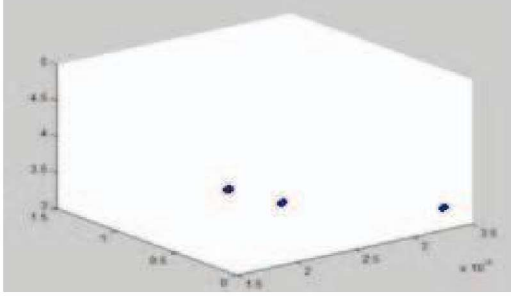


Fig.5. Optimal Solutions for NSGA-II Approach [4]

Using solution-I, the test cases chosen for optimal suite are given in Table-V.

TABLE V
OPTIMAL SUITE USING NSGA-II
(SOLUTION-I)

Sr#	Path	Time	Coverage	Faults
2	6	0.001796	0.653	1
5	3	0.001686	0.647	2
6	3	0.001752	0.647	2
11	3	0.001967	0.647	2
15	2	0.002361	0.8	1
17	3	0.00177	0.647	2

Test suite has been reduced by 68 percent (approx) but resultant test suite is unable to reveal the faults for paths 1, 4 and 5. Thus this solution is not safe and is imprecise because those test cases that reveal the same faults are included in the test suite. 2 out of 5

Using solution-II, the test cases chosen for optimal suite are given in Table-VI.

TABLE VI
OPTIMAL SUITE USING NSGA-II
(SOLUTION-II)

Sr#	Path	Time	Coverage	Faults
4	3	0.002508	0.647	2
9	3	0.002552	0.647	3
10	3	0.002549	0.647	2

Resultant test suite has reduced by 84 percent approximately but comprises test cases that cannot reveal faults for path 1, 2, 4, 5 and 6. 3 out of 5

Using solution-III, test cases chosen for optimal suite are given in Table-VII.

TABLE VII
OPTIMAL SUITE USING NSGA-II
(SOLUTION-III)

Sr#	Path	Time	Coverage	Faults
1	4	0.003704	0.497	2
12	5	0.003691	0.5	2
13	1	0.003644	0.317	4

Resultant test suite has reduced by 84 percent approximately but is unable to reveal faults for paths 2, 3 and 6. This approach has been successful to explore only 4 faults out of 5 seeded faults.

Thus all suitable optimal solutions generated using NSGA-II has not been able to traverse all paths. This reflects that NSGA-II is not safe and precise approach.

C. Multi Objective Particle Swarm Optimization(MOPSO)

Multi Objective Particle swarm optimization was proposed by Coello et al in 2004 [40]. We have implemented MOPSO using objective functions defined through Eq.1-3.Pareto front generated using MOPSO is given in [4].

Suitable solutions generated after analyzing the data file are given in Fig.6.

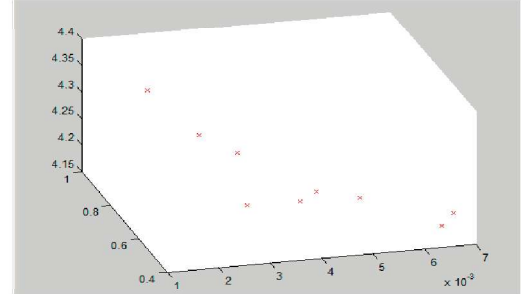


Fig.6. Suitable Solutions MOPSO Approach [4]

Using solution-I (range of coverage 0.8 to 1 and Time from 0.001 to 0.003) the test cases chosen for optimal suite are given in Table-VIII.

TABLE VIII
OPTIMAL SUITE USING MOPSO
(SOLUTION-I)

Sr#	Path	Time	Coverage	Faults
3	2	0.002822	0.8	2
15	2	0.002361	0.8	1
16	2	0.002569	0.8	2

Resultant test suite has been reduced by 84 percent approximately but comprises test case that can not reveal faults for paths 1, 3, 4, 5 and 6. Using solution-II (range of coverage 0.6 to 0.79 and Time from 0.003 to 0.004), the test cases selected in optimal suite are given in Table-IX.

TABLE IX
OPTIMAL SUITE USING MOPSO
(SOLUTION-II)

Sr#	Path	Time	Coverage	Faults
18	3	0.003093	0.647	3

Resultant test suite has been drastically reduced by 99 percent approximately but comprises test case that can not reveal faults for paths 1, 2, 4, 5 and 6.

Other solutions in range of coverage 0.6 to 0.79 and Time from 0.004 to 0.005 and in range of coverage 0.4 to 0.6 and Time from 0.005 to 0.007 contain empty set and does not contain any test cases for our research problem. This approach

has been successful to explore 3 out of 5 seeded faults. Thus, it can be concluded that MOPSO is not safe test suite reduction approach due to its inability to traverse all the paths.

D. Multi Objective Fuzzy based Optimization [4]

Fuzzy based approach [4] provides a single solution after implementation of safe reduction criterion on optimal results and is given in Table X.

TABLE X
OPTIMAL SUITE USING FUZZY LOGIC [4]

Sr#	Path	Time	Coverage	Faults
1	4	0.003704	0.497	2
2	6	0.001796	0.653	1
4	3	0.002508	0.647	2
12	5	0.003691	0.5	2
16	2	0.002569	0.8	2
19	1	0.007112	0.317	4

This approach has been successful to reduce the test suite by 68 percent and contains test cases for all paths and can reveal faults for each path. Thus or approach is safe and precise and there are no redundant test cases in the optimal suite.

IV. CONCLUSION AND FUTURE DIRECTIONS

We have studied the CI based approaches on quadratic equation problem. We have found that these approaches can reduce the test suite size but are unable to provide safe reduction. We conclude that safe test suite reduction is not possible due to non existence of any constraint that can restrict solutions to contain test cases from all paths of control flow graph.

We have found that CI based approaches except fuzzy logic require some methodology for safe reduction. Any reduction that is unsafe is meaningless and does not provide adequate testing. We will implement our approach on sufficiently large test suite to validate our conclusion about the CI based approaches. We will further use neural networks for test suite optimization to assess possibility of safe reduction with neural networks. Another future direction of our work is to define some methodology that could provide safe reduction after implementation of safe reduction criterion on CI based approaches.

REFERENCES

- [1] Roger S. Pressman, Software Engineering, A Practitioner's Approach, 7th ed, Mc Graw Hill, 2010.
- [2] Robert V. Binder, Testing Object Oriented Systems: Models, Patterns, and Tools, 5th ed, Addison Wesley, 2004.
- [3] S.Biswas and R.Mall, Regression Test Selection Techniques-A Survey, In Informatica, 2011, Vol 35, 289-321.
- [4] A.A. Haider, A. Nadeem, S. Rafiq, "Multiple Objective Test Suite Optimization: A Fuzzy Logic Based Approach", In: Journal of intelligent and Fuzzy Systems, DOI 10.3233/IFS-131045.
- [5] S.Yoo, M. Harman "Pareto Efficient Multi-Objective Test Case Selection" in proceedings of ISSSTA '07, July 9-12, London, U.K.
- [6] A.A. Haider, S. Rafiq, A. Nadeem, Test suite optimization using fuzzy logic, in: proceedings of 8th International conference on emerging technologies (ICET), 2012.
- [7] S. Yoo, M. Harman, Regression Testing Minimisation, Selection and Prioritisation: A Survey, In Journal of Software Testing, Verification and Reliability, 2007, 00:1-7.
- [8] S.Biswas and R.Mall, Regression Test Selection Techniques-A Survey, In Informatica, 2011, Vol 35, 289-321.
- [9] Andries P. Engelbrecht, Computational Intelligence: An Introduction, 2nd ed., John Wiley & Sons, 2007.
- [10] M.Kumar, A.Sharma, R.Kumar, Optimization of Test Cases using using soft computing techniques: A critical review, in WESEAS transactions on information science and applications, vol. 8, 2011.
- [11] A.A. Haider, A. Nadeem, S.Rafiq, On the Fly Test Suite Optimization with FuzzyOptimizer, in: proceedings of frontiers of information technology (FIT), 2013.
- [12] M. Melanie, An Introduction to Genetic Algorithms, 5th ed, Massachusetts Institute of Technology, 1999.
- [13] A. Kaur, S. Goyal, A Genetic Algorithm for Regression Test Case Coverage Prioritization Using Code Coverage, International Journal on Computer Science and Engineering, Vol. 3 No. 5, May-11, pp1839-1847.
- [14] A. Kaur, S. Goyal, A Genetic Algorithm for Fault based Regression Test Case Prioritization, International Journal of Computer Applications, Volume 32- No.8, Oct 2011.
- [15] S. Nachiyappan, A. Vimaladevi, C.B. SelvaLakshmi, An Evolutionary Algorithm for Regression Test Suite Reduction, in proceedings of the International Conference on Communication and Computational Intelligence, India.27 - 29 December, 2010, pp. 503-508.
- [16] R. Krishnamoorthi, S.A. Sahaaya Arul Mary, Regression Test Suite Prioritization using Genetic Algorithm, International Journal of Hybrid Information Technology, Vol.2, No.3, July, 2009.
- [17] D. Teodorovic, Bee Colony Optmization (BCO), In C.P Lim, S. Dehuri (eds) L.C Jain, Innovations in Swarm Intelligence, 2009, pp 39-60.
- [18] M. Dorigo, T. Stutzle, Ant Colony Optimization, Massachusetts Institute of Technology, 2004.
- [19] M. Clerc, Particle Swarm Optimization, ISTE (2006), 256 pp.
- [20] Kaur, S. Goyal "A Bee Colony Optimization Algorithm for Fault Coverage Based Regression Test Suite Prioritization" International Journal of Advanced Science and Technology, Vol. 29, April, 2011
- [21] B.Suri, S. Singhal "Implementing Ant Colony Optimization for Test Case Selection and Prioritization" International Journal on Computer Science and Engineering (IJCSSE), Vol. 3 No. 5 May 2011
- [22] K.Vivekanandan, G.Keerthi Lakshmi "Improving Regression Testing through Modified Ant Colony Algorithm on a Dependency Injected Test Pattern" International Journal of Computer Science Issues, Vol. 9, Issue 2, No 2, March 2012
- [23] A. Kaur, D. Bhatt "Particle Swarm Optimization with Cross-Over Operator for Prioritization in Regression Testing", International Journal of Computer Applications (0975 - 8887), Volume 27- No.10, August 2011
- [24] A. Kaur, D. Bhatt "Hybrid Particle Swarm Optimization for Regression Testing", International Journal on Computer Science and Engineering, Vol. 3 No. 5 (May-11), pp1815-1824, ISSN: 0975-3397.
- [25] L. S. de Souza, P. B. C. de Miranda, R. B. C. Prudencio, F. A. Barros, A Multi-Objective Particle Swarm Optimization for

- Test Case Selection Based on Functional Requirements Coverage and Execution Effort, In Proceedings of 23rd International Conference on Tools with Artificial Intelligence, 2011.
- [26] R. Sehrawat, V. Srivastava, I. Mangal "A Neuro-Genetic Algorithm for Selection of Test Suites in Regression testing" International Journal of Computer Application Issues2, Vol-1 (Feb-12), ISSN: 2250-1797.
 - [27] A. da Silva Simao, de R.F. Mello, L.J. Senger, A Technique to Reduce the Test Case Suites for Regression Testing Based on a Self-Organizing Neural Network Architecture, in : 30th International computer software and application conference, 2006.
 - [28] Z. Xu, K. Gao, T.M. Khoshgoftaar, Application of fuzzy expert system in test case selection for system regression test, in: Proc. IRI, pp. 120-125, 2005.
 - [29] C. Malz, N. Jazdi, P. Gohner, Prioritization of test cases using software agents and fuzzy logic, in: proceedings of Fifth International Conference on Software Testing, Verification and Validation, 2012.
 - [30] H. Bhasin, S. Gupta, M. Kathuria, Regression Testing using Fuzzy Logic, in: International Journal of Computer Science and Information Technology, Vol. 4, 2013.
 - [31] M. Kumar, A. Sharma, R. Kumar, Towards Multi-Faceted Test Cases Optimization, Journal of Software Engineering and Applications, 2011, 4, 550-557.
 - [32] M. Kumar, A. Sharma, R. Kumar, Multi faceted measurement framework for test case classification and fitness evaluation using fuzzy logic based approach, in: Chiang Mai Journal of Science, Issue 39, Vol 3, 2012.
 - [33] G. Kumar, P. Kumar, Software testing optimization through test suite reduction using fuzzy clustering, in: CSI Transactions in ICT, 2013.
 - [34] A. Konak, D. W. Coit, A. E. Smith, Multi-objective optimization using genetic algorithms: A tutorial, in Reliability Engineering and System Safety 91 (2006) 992–1007.
 - [35] Carlos A. Coelo Coelo, G. T. Pulido, M.S. Lachuga " Handling Multiple Objectives with Particle Swarm Optimization" in IEEE Transactions on evolutionary computation, Vo 8, no 3, 2004.
 - [36] J. Bible, G. Rothermel, D. S. Rosenblum, A Comparative Study of Coarse and Fine Grained Safe Regression Test Selection Techniques, ACM Transactions on Software Engineering and Methodology, Vol. 10, No. 2, April 2001, Pages 149–183.
 - [37] C. M. Fonseca, P.J. Fleming, "Genetic Algorithms for Multi Objective Optimization: Formulation, Discussion and Optimization", in Proceedings of fifth international conference on genetic algorithms, 1993.
 - [38] <http://www.mathworks.com/matlabcentral>.
 - [39] N. Srinivas. K. Debb "Multi objective function optimization using non dominated sorted genetic algorithms", Evolutionary Computation, Vol 2, no 3, pp 221-248, 1995.
 - [40] Carlos A. Coelo Coelo, G. T. Pulido, M.S. Lachuga " Handling Multiple Objectives with Particle Swarm Optimization" in IEEE Transactions on evolutionary computation, Vo 8, no 3, 2004.